ElasticSearch & Kibana

- ElasticSearch və Kibana bir-birinə çox sıx bağlı **search və analytics ekosistemi**nin əsas hissələridir.

Komponent	Rol
Elasticsearch	Məlumatları saxlamaq, axtarmaq və analiz etmək üçün əsas **search engine**
Logstash	Gələn məlumatları **toplayır, emal edir və Elasticsearch-ə göndərir**
Kibana	Elasticsearch-dəki məlumatları **vizuallaşdırmaq və analiz etmək** üçün interfeys
Beats	Yüngül agentlərdir — müxtəlif mənbələrdən log və metrikləri toplayıb Logstash/Elasticsearch-ə göndərir

Elasticsearch nədir?

- **Elasticsearch** — böyük həcmli məlumatlar üzərində **sürətli axtarış, filtr, analiz və sorğular** aparmaq üçün yaradılmış **distributed NoSQL search engine**-dir.Normalda biz bazaya hər hansı bir sorğu göndərsək o sorğuya görə bazada axtarış edir.Amma ElasticSearch bunları optimallaşdırılmış şəkildə özündə saxlayır.

Məlumatlar harada və necə saxlanır?

- ElasticSearch-in içində İndex deyilən ‘məlumat saxlama yeri’ var.
- Bu SQL-dəki database kimidir.

Məsələn:

- Sən Users məlumatını saxlamaq istəyirsənsə ElasticSearch də Users index i yaradırsan.

İndex nədir?

- Index –Documentlərdən ibarətdir.
- Hər document özündə 1 məlumatı saxlayır.
- Və datalar Json formatda olur.

```
{
  "id": 1,
  "name": "Valeh",
  "city": "Baku",
  "skills": ["C#", "Golang"]
}
```

Necə işləyir?

ID	Mətn
1	Valeh Golang öyrənir
2	Golang çox sürətlidir
3	Valeh Elasticsearch istifadə edir

Bu bizim bazamızdır.Biz burda “Valeh” sözünü axtarmaq istəyirik.
🐢 **SQL-də necə olur:**
SQL hər sətri oxuyur və soruşur:
Sətir 1-də “Valeh” var? ✅
Sətir 2-də “Valeh” var? ❌
Sətir 3-də “Valeh” var? ✅
Yəni hər sətri **başa qədər oxuyur** → bu **yavaşdır**, xüsusilə milyonlarla sətr olanda.

ElasticSearch necə edir?

Elasticsearch bu işi **əvvəldən hazırlayır** — yəni sən məlumat əlavə edəndə, o mətni **sözlərə bölür** və **xəritə yaradır.**

Söz	Hansi sənədlərdə var
valeh	[1, 3]
golang	[1, 2]
öyrənir	[1]
sürətlidir	[2]
elasticsearch	[3]
istifadə	[3]
edir	[3]

- 🔍 **3. Axtarış necə işləyir**
- İndi sən “Valeh” yazanda:
- Elasticsearch həmin **inverted index cədvəlinə baxır**
- Görür ki “valeḥ” → [1, 3]
- Dərhal sənə ID 1 və 3-ü qaytarır
- **Heç bütün sənədləri oxumur!**
  Yəni bu **birbaşa indeksdən cavab tapır.**

- Adi indeks

Document-> Words yəni sənədin içində hansı sözlər var

- Tərs indeks isə:

Word->Documents yəni sözə görə sənədləri tapırıq.

Necə yaradılır?

- Məsələn sən belə sənəd əlavə edirsən:

{ "text": "Valeh Elasticsearch öyrənir" }

- Elasticsearch bunu belə emal edir:

Analyzer-mətni kiçik hərflərə çevirir ->["valeh", "elasticsearch", "öyrənir"]

Tokenizer-cümləni sözlərə bölür

İnverted İndex-bu sözlər üçün xəritə yaradır.

```
valeh → [document 1]
elasticsearch → [document 1]
öyrənir → [document 1]
```

Shard nədir?

- **Shard** — index-in bir hissəsidir, yəni o index-dəki məlumatın bir alt bölməsidir.
- Hər shard öz-özünə mini index dir.
- Hər shard Elasticsearch cluster-də fərqli node-da yerləşə bilir.

Cluster nədir?

- **Cluster** — bu, Elasticsearch sisteminin “bütöv beyni” kimidir.
  O, **bir və ya bir neçə node-dan (serverdən)** ibarətdir və **bütün verilənlər bazasını idarə edən ümumi sistem** kimi işləyir.
- Bir cluster-ın əsas məqsədi:
- Məlumatları saxlamaq,
- Axtarış (search) və analiz sorğularını yerinə yetirmək,
- Yüksək əlçatanlıq (high availability) və paylanmış hesablama təmin etməkdir.

- **Məsələn:**
- Təsəvvür et sən “TravelTrickApp” üçün Elasticsearch qurursan.
  Sənin bütün “requests”, “destinations”, “users” və s. məlumatların Elasticsearch-də saxlanılır.
  Bunların hamısını **idarə edən sistem — sənin Elasticsearch cluster-ındır**.
- Cluster-a bir ad verilir (default: elasticsearch), və o adla bütün node-lar bir-birini tanıyır.

Node nədir?

- **Node** — bu, Elasticsearch cluster-ının daxilində çalışan **bir server (və ya instans)** deməkdir.
  Yəni, **cluster → bir çox node-dan ibarət olur**.
- Hər bir node, **müəyyən rola malikdir** və bu rollar onun cluster daxilində nə iş gördüyünü müəyyən edir.

- Node lar özüdə işləmə funksiyasına görə 4 yerə bölünür:

1. Master Node
2. Data Node
3. Ingest Node
4. Coordinating Node

1 Master Node → “Direktor”

- **İşləmə prinsipi:**
- Şirkətin (cluster-in) idarəçisidir.
- Kim hansı otaqda (node-da) işləyəcək, fayllar (shard-lar) necə bölünəcək — bunlara qərar verir.
- Yəni **idarəetmə** işini görür, amma **məlumat saxlamır**.
- **Qısaca:**
- Master node = idarəçi → qərar verir, amma data ilə işləməz.

💾 2️⃣ Data Node → “İşçi”

- **İşləmə prinsipi:**
- Real datanı **özündə saxlayan** və **axtarış** əməliyyatlarını yerinə yetirən node-dur.
- İstifadəçilər axtarış edəndə, nəticəni məhz data node-lar tapır və qaytarır.
- **Qısaca:**
- Data node = məlumatı saxlayır və sorğularla işləyir.

3 Ingest Node → “Katib”

- **İşləmə prinsipi:**
- Data şirkətə (Elasticsearch-ə) daxil olanda onu **hazırlayan** və **formatlayan** node-dur.
- Məsələn, sən “log” göndərirsən, o “katib” həmin log-u təmizləyir, bölür və sistemə hazır şəkildə ötürür.
- **Qısaca:**
- Ingest node = datanı içəri girməzdən əvvəl səliqəyə salır.

4 Coordinating Node → “Kuryer / Yönləndirici”

- **İşləmə prinsipi:**
- Müştəri (API və ya Kibana) sorğu göndərir, bu node həmin sorğunu qəbul edir.
- Həmin sorğunu **uyğun data node-lara** göndərir və cavabları **toplayıb birləşdirir.**
- **Qısaca:**
- Coordinating node = sorğuları qəbul edib yönləndirən vasitəçidir.

Sadə misal:

Deyək ki sən “user: Valeh” axtarırsan:

Sorğu **coordinating node**-a gəlir.

O, “Valeh” məlumatının **hansı data node-da** olduğunu öyrənir.

Sorğunu həmin **data node-lara** göndərir.

Data node məlumatı tapır və qaytarır.

Coordinating node nəticələri **birləşdirib sənə** göndərir.